Étude du calcul des composantes connexes avec Apache Spark

Analyse des notebooks Scala et Python, implémentations RDD / DataFrame, et interprétation expérimentale

Projet : Finding connected components in graph	**Livrables analysés :** un notebook Scala Zeppelin et un notebook Python/PySpark
Références : papier CCF (Connected Component Finder) et consignes du projet	**Résultats intégrés :** benchmarks Scala et Python fournis par l'étudiant

1. Contexte et objectif du projet

Le sujet impose de comprendre l'algorithme décrit dans le papier CCF, puis de le recoder dans Spark en utilisant les deux API principales : **RDD** et **DataFrame**, avec des versions en **Scala** et en **Python**. L'évaluation attend une implémentation correcte, une analyse expérimentale de la scalabilité et une discussion des points forts et limites de chaque approche.

Le papier de référence propose une stratégie itérative de propagation du plus petit identifiant au sein d'une composante connexe. Dans le projet réalisé ici, cette idée est reformulée sous forme de **propagation de labels** : chaque nœud transporte un label, reçoit les labels de ses voisins, puis adopte le minimum jusqu'à stabilisation.

2. Analyse des notebooks fournis

2.1 Notebook Scala

Le notebook Scala contient une progression claire en paragraphes Zeppelin. Il commence par les imports Spark SQL, RDD et StorageLevel, puis construit un petit graphe de test composé de deux composantes : *A-B-C-D-E* et *F-G-H*. Ensuite, il définit **makeUndirected**, qui nettoie les arêtes, supprime les self-loops et ajoute les arêtes inverses pour transformer le graphe en graphe non orienté.

La partie centrale du notebook introduit ensuite deux fonctions principales : **runCCFRDD** et **runCCFDataFrame**. La première implémente une propagation de labels à l'aide des transformations RDD (*join*, *reduceByKey*, *leftOuterJoin*). La seconde reprend la même logique via des jointures et agrégations DataFrame, avec persistance mémoire/disque et *checkpointing* pour maîtriser la profondeur de lineage.

Enfin, le notebook Scala définit un générateur de graphes synthétiques par chaînes disjointes, puis deux fonctions de benchmark, une pour RDD et une pour DataFrame, avant de produire un tableau final contenant le nombre de nœuds, le nombre d'arêtes, le temps d'exécution et le nombre d'itérations.

2.2 Notebook Python

Le notebook Python suit pratiquement la même structure logique, mais ajoute en amont une phase d'initialisation de l'environnement PySpark : désinstallation/réinstallation de la bonne version de PySpark, configuration de **JAVA_HOME**, création de la **SparkSession** et réglage du niveau de

parallélisme SQL. Il contient également une version plus optimisée des deux algorithmes, nommées **run_ccf_rdd_fast** et **run_ccf_dataframe_fast**.

La fonction **run_ccf_rdd_fast** introduit explicitement le partitionnement des RDD de paires, la persistance en *MEMORY_AND_DISK*, et une logique de co-partitionnement pour limiter les coûts de shuffle. La fonction **run_ccf_dataframe_fast** repartitionne le graphe et les labels, utilise un *checkpoint* périodique, et prévoit un seuil de *broadcast* lorsque la table des labels est suffisamment petite.

Le notebook Python réutilise ensuite le même générateur de graphes synthétiques et le même protocole de benchmark que la version Scala, ce qui permet une comparaison pertinente entre les quatre configurations : Scala RDD, Scala DataFrame, Python RDD et Python DataFrame.

3. Explication du code

3.1 Préparation du graphe

Dans les deux langages, la première étape consiste à construire un DataFrame d'arêtes à deux colonnes : **src** et **dst**. La fonction de préparation réalise trois opérations essentielles : filtrer les valeurs nulles, supprimer les boucles de type *(u, u)*, et rendre le graphe non orienté en ajoutant l'arête inverse *(v, u)* pour chaque arête *(u, v)*.

Le petit graphe de test permet ensuite de vérifier rapidement la correction. Après convergence, la sortie attendue est : **A→A, B→A, C→A, D→A, E→A** pour la première composante, et **F→F, G→F, H→F** pour la seconde.

3.2 Implémentation RDD

La version RDD est la plus proche de l'esprit MapReduce du papier CCF. Le graphe est converti en RDD de paires, puis re-clé par destination pour faciliter la propagation des labels depuis les voisins vers le nœud courant. Chaque nœud commence avec son propre identifiant comme label initial. À chaque itération, les labels des voisins sont propagés, réduits par minimum, puis fusionnés avec les labels courants.

Concrètement, trois opérations dominent chaque itération : **join** pour associer un voisin à son label courant, **reduceByKey** pour garder le plus petit label reçu, et **leftOuterJoin** pour mettre à jour le label du nœud. L'algorithme s'arrête quand le nombre de nœuds dont le label change devient nul.

L'intérêt principal de la version RDD est son contrôle très fin sur les données et sur la logique itérative. Elle s'aligne bien avec la formulation MapReduce du papier, mais son code reste plus bas niveau et moins déclaratif.

3.3 Implémentation DataFrame

La version DataFrame reconstruit la même logique avec des opérations relationnelles. Le DataFrame des labels contient deux colonnes : **node** et **label**. À chaque itération, le graphe est joint avec la table des labels pour associer à chaque nœud les labels de ses voisins, puis un **groupBy** avec **min** calcule le plus petit label voisin observé.

Le nouveau label est ensuite défini par le minimum entre le label courant et ce minimum voisin. Pour éviter une explosion de lineage, le notebook utilise la persistance et le **checkpoint local**. Dans la version Python optimisée, le graphe et les labels sont repartitionnés, le checkpoint n'est pas forcé à chaque itération, et un broadcast join peut être utilisé lorsque la table de labels est petite.

Cette approche est plus élégante du point de vue SQL/Spark Structured API, mais elle introduit plus de coût sur les jointures et agrégations répétées. C'est précisément ce que l'analyse expérimentale va

montrer pour Scala, tandis qu'en Python les résultats révèlent un comportement différent.

4. Protocole expérimental

Les deux notebooks génèrent des graphes synthétiques constitués de plusieurs chaînes disjointes. Cette famille de graphes est volontairement simple : elle permet de maîtriser le nombre de composantes, de conserver une structure sparse, et de prévoir facilement le comportement de propagation. Les tailles testées sont : **50, 100, 200 et 400 nœuds**, avec un nombre d'arêtes cohérent avec une structure en chaînes.

Pour chaque taille, quatre indicateurs sont collectés : le nombre de nœuds, le nombre d'arêtes, le temps d'exécution et le nombre d'itérations jusqu'à convergence. Cette stratégie permet de comparer non seulement les vitesses relatives, mais aussi la stabilité logique des implémentations.

5. Résultats expérimentaux

5.1 Résultats Scala

num_nodes	num_edges	rdd_time_ms	rdd_iterations	df_time_ms	df_iterations
50	45	8454	10	17572	10
100	95	15034	20	33059	20
200	190	15044	20	33755	20
400	390	33209	40	74742	40

Tableau 1 — Benchmarks Scala (RDD vs DataFrame)

5.2 Résultats Python

num_nodes	num_edges	rdd_time_ms	rdd_iterations	df_time_ms	df_iterations
50	45	17415	10	15259	10
100	95	31679	20	20790	20
200	190	30954	20	19243	20
400	390	61048	40	35131	40

Tableau 2 — Benchmarks Python (RDD vs DataFrame)

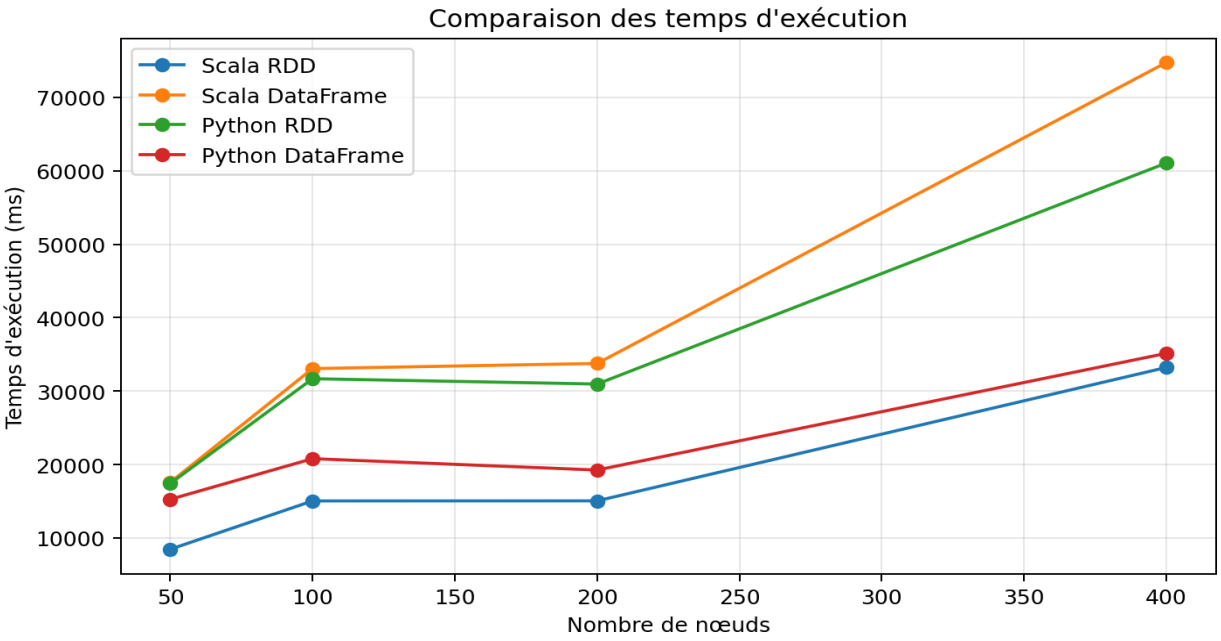


Figure 1 — Comparaison des temps d'exécution entre les quatre implémentations

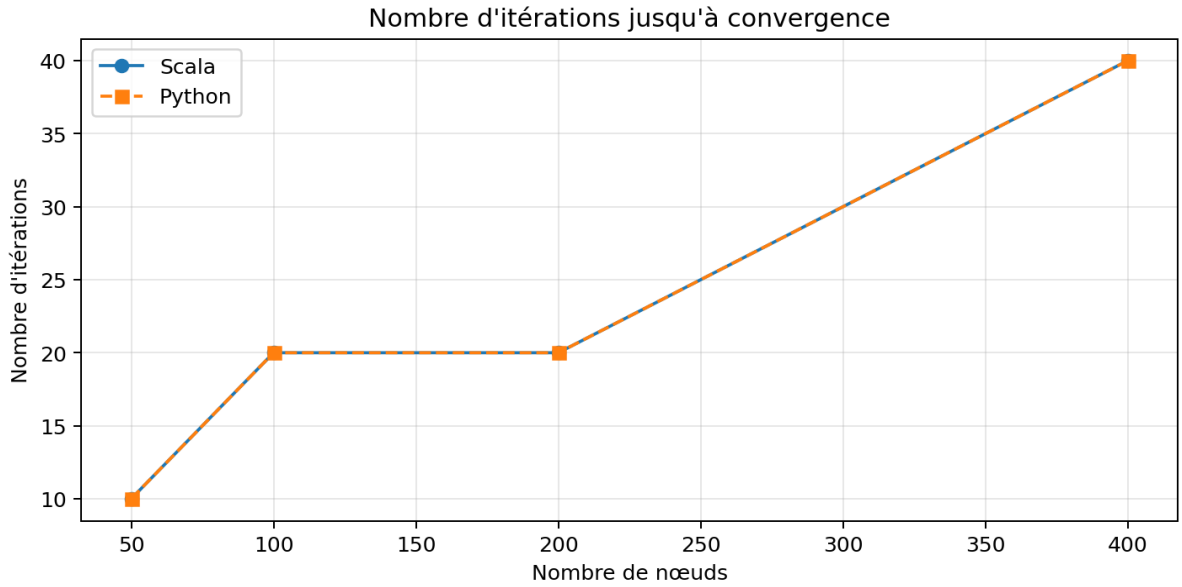


Figure 2 — Nombre d'itérations jusqu'à convergence

6. Analyse des résultats

6.1 Tendances générales

Le premier constat important est que le **nombre d'itérations** est identique entre Scala et Python, ainsi qu'entre les versions RDD et DataFrame pour une même taille de graphe. Cela signifie que les quatre implémentations convergent vers la même solution logique, et que les écarts observés relèvent principalement du coût d'exécution, pas d'un comportement algorithmique différent.

Un second constat est que le nombre d'itérations augmente avec la taille du graphe : 10 itérations pour 50 nœuds, 20 pour 100 et 200 nœuds, puis 40 pour 400 nœuds. Cela est cohérent avec l'idée de propagation de labels sur des chaînes relativement longues : plus le diamètre utile d'une composante est élevé, plus le minimum met du temps à traverser toute la chaîne.

6.2 Scala : avantage clair aux RDD

Dans les résultats Scala, la version **RDD** est systématiquement plus rapide que la version **DataFrame**. Par exemple, pour 400 nœuds, Scala RDD met environ **33 209 ms** contre **74 742 ms** pour Scala DataFrame. Le ratio reste proche de 2× sur l'ensemble des tailles testées. Cela indique que, dans ce contexte, les RDD sont mieux adaptés à la nature itérative et graphe-orientée du problème.

Cette observation est cohérente avec la structure du code Scala. La version RDD manipule directement des paires clé-valeur et utilise des transformations proches de MapReduce, alors que la version DataFrame enchaîne des jointures et des agrégations plus coûteuses à chaque itération. En Scala, le surcoût de l'API relationnelle domine clairement pour ce type de charge.

6.3 Python : avantage à DataFrame

Le comportement en Python est différent : la version **DataFrame** est plus rapide que la version **RDD** sur toutes les tailles testées. Pour 400 nœuds, Python RDD atteint **61 048 ms**, alors que Python DataFrame termine en **35 131 ms**. Cette inversion par rapport à Scala s'explique probablement par le coût supplémentaire des transformations RDD en PySpark, où les opérations traversent davantage la frontière Python/JVM.

Le notebook Python contient aussi une optimisation plus poussée pour DataFrame : repartitionnement explicite, persistance, checkpoint périodique et seuil de broadcast. Ces raffinements réduisent le coût des jointures répétées et rendent l'approche DataFrame particulièrement compétitive en PySpark.

6.4 Lecture transversale Scala vs Python

Si l'on compare les quatre implémentations ensemble, on obtient une lecture très intéressante. En **Scala**, l'approche la plus efficace est RDD, car elle reste proche de l'algorithme bas niveau et bénéficie d'une exécution directe dans l'écosystème JVM. En **Python**, au contraire, l'approche DataFrame prend l'avantage, car elle laisse davantage le moteur Spark optimiser les opérations côté JVM, tandis que les transformations RDD PySpark sont plus pénalisées.

Cela montre qu'une même idée algorithmique ne se comporte pas de façon identique selon l'API et le langage. Le choix entre RDD et DataFrame ne dépend donc pas uniquement du problème, mais aussi de l'environnement d'exécution et du coût de l'interface langage-moteur.

7. Points forts et limites

Points forts :

- Les quatre implémentations convergent correctement vers la même solution, ce qui valide la logique générale de propagation de labels.
- La version Scala RDD est très fidèle à l'esprit MapReduce du papier CCF.
- La version Python DataFrame montre qu'une API plus haut niveau peut devenir la meilleure option lorsque le langage d'appel introduit un surcoût important sur les RDD.
- Le protocole expérimental est cohérent : mêmes graphes, mêmes métriques, même logique d'arrêt.

Limites :

- Les graphes testés restent de petite taille et très structurés (chaînes disjointes). Les conclusions doivent donc être nuancées pour des graphes plus denses ou plus irréguliers.
- Les temps mesurés incluent l'overhead de l'environnement notebook, qui peut être non négligeable sur des tailles modestes.
- Les performances observées en local ou en environnement pédagogique ne reflètent pas forcément le comportement exact sur un cluster distribué de grande taille.

- Les versions DataFrame ont nécessité des mécanismes de persistance et de checkpoint pour rester stables, ce qui complexifie légèrement leur maintenance.

8. Conclusion

Les notebooks analysés constituent un travail cohérent et bien structuré pour porter le problème des composantes connexes vers Spark. Le pipeline est clair : préparation du graphe, validation sur un petit exemple, implémentations RDD et DataFrame, génération de graphes synthétiques, puis mesure expérimentale. Du point de vue algorithmique, la solution par propagation de labels est simple, robuste et adaptée au cadre pédagogique du projet.

L'analyse des résultats permet de tirer une conclusion nuancée mais forte : **en Scala, RDD est la meilleure option pour ce problème**, tandis qu'**en Python, DataFrame devient plus performant**. Autrement dit, l'API optimale dépend à la fois de la nature itérative du calcul et du langage utilisé pour piloter Spark. Cette conclusion enrichit le rapport au-delà d'une simple implémentation technique, car elle met en évidence un vrai enseignement d'architecture logicielle.

Pour aller plus loin, il serait pertinent de tester des graphes plus grands, plus denses, et d'ajouter éventuellement une comparaison avec des bibliothèques spécialisées comme GraphX ou GraphFrames. Néanmoins, au vu des consignes du projet, le travail réalisé couvre bien les attentes : compréhension de l'algorithme, double implémentation, et étude de performance structurée.

Annexe courte — fragments clés du code

```
Scala / makeUndirected:
def makeUndirected(dfEdges: DataFrame): DataFrame = {
val cleaned = dfEdges.filter(col("src").isNotNull && col("dst").isNotNull)
.filter(col("src") =!= col("dst")).distinct()
val reversed = cleaned.select(col("dst").as("src"), col("src").as("dst"))
cleaned.union(reversed).distinct()
}
```

```
Python / RDD optimisé:
def run_ccf_rdd_fast(...):
undirected_rdd = df_undirected.select("src","dst").rdd.map(...)
graph_by_dst_rdd = undirected_rdd.map(lambda x: (x[1], x[0]))
min_neighbor_labels = propagated.reduceByKey(...)
next_labels_rdd = labels_rdd.leftOuterJoin(min_neighbor_labels)...
```

```
Python / DataFrame optimisé:
def run_ccf_dataframe_fast(...):
df_graph = df_undirected.select("src","dst").repartition(num_partitions, "dst")
propagated = df_graph.join(labels_for_join, df_graph["dst"] == labels_for_join["node"])
min_neighbor_labels = propagated.groupBy("node").agg(F.min("neighbor_label"))
next_labels_df = ...
```